

Projet Modélisation et Vérification des Systèmes Concurrents

CREZE Tudwall

12 novembre 2025

1 Introduction

Dans le cadre du travail attendu pour le DM, j'ai choisi de le réaliser en Python. Mon travail se retrouve découpé entre 4 fichiers :

- **TransitionSystem.py** : Définition de la classe d'un système de transition.
- **verifier_invariant.py** : Fichier contenant l'implémentation de l'algorithme de vérification de l'invariant (Algorithme 1) et de la procédure de visite.
- **properties.py** : Définition des fonctions de satisfaction ($s \models \Phi$) pour les différentes propriétés à valider pour chaque exemple.
- **invariant_phi_test.py** : Fichier d'exemple contenant les systèmes de transition et l'exécution de leur vérification associée.

Pour obtenir le résultat, on exécute simplement `python ./invariant_phi_test.py`.

Dans le fichier test se trouve la déclaration de 2 objets de type `TransitionSystem` :

1. Un représentant la concurrence entre 2 programmes contrôlée par un sémaphore, dont la propriété de sûreté que nous souhaitons vérifier est l'**Exclusion Mutuelle** $\neg(C1 \wedge C2)$.
2. L'autre, un système de transition simple représentant un compteur entre 0 et 4, dont la propriété à valider est **MAX** (le compteur ne doit pas atteindre 4).

2 Définition d'un Système de Transition

Le choix des structures de données en Python est primordial pour garantir la complexité temporelle optimale de l'algorithme.

- **États et propositions** : Les attributs `self.S` (*States*), `self.I` (*InitialState*) et `self.Prop` (Propositions) représentent les états et les propositions validées en eux ainsi que la déclaration des états initiaux.
- **Relation de Transition** (`self.Post`) : La relation $\rightarrow$ est modélisée par un dictionnaire (`Dict`) qui stocke l'ensemble des successeurs ($Post(s)$) pour chaque état s . Ce choix garantit un accès en temps constant $O(1)$ aux successeurs, essentiel pour le parcours de graphe (DFS).
- **Fonction de Labellisation** (`self.L`) : La fonction L est également implémentée par un dictionnaire. L'accès à l'ensemble des propositions atomiques vérifiées par un état s (`self.L.get(s)`) se fait en $O(1)$, permettant une vérification rapide de la satisfaction $s \models \Phi$ à chaque étape de l'algorithme.

Les ensembles (`Set`) sont utilisés à l'intérieur de ces dictionnaires et pour la gestion des états accessibles (R) pour leurs propriétés d'unicité et la rapidité des opérations d'ensemble.

3 Système de Transition 1 : Sémaphore Binaire (Exclusion Mutuelle)

Ce ST modélise deux processus concurrents contrôlés par un sémaphore binaire. La modélisation utilise des états simplifiés : **N** (Non-Critique), **P** (Pré-Critique/Attente), et **C** (Critique).

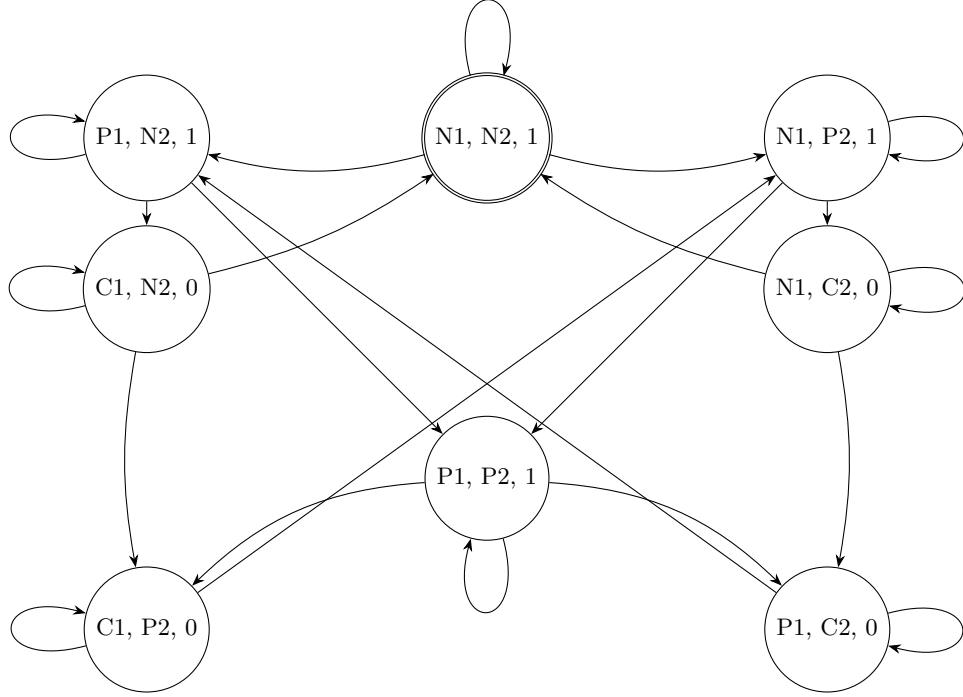


FIGURE 1 – Système de Transition du Sémaphore Binaire restreints aux états accessibles par l'état initial par soucis de simplicité (le code contient les autres états/transitions)

4 Système de Transition 2 : Compteur (0 à 4)

Ce ST modélise un simple compteur séquentiel, utilisé pour vérifier l'accès à l'état **MAX**.

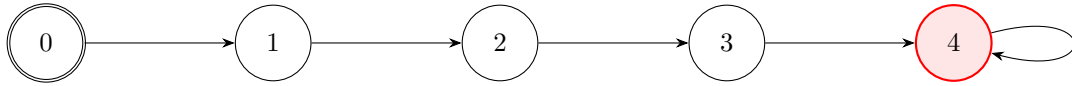


FIGURE 2 – Système de Transition Compteur (de 0 à 4). L'invariant $\Phi : \neg \mathbf{MAX}$ est violé car l'état 4 est accessible. Le chemin de contre-exemple est $0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4$.

5 Conclusion

Ce projet a permis de concrétiser l'algorithme de vérification d'invariant (Algorithme 1) pour les systèmes de transition finis. L'implémentation en **Python** a mis l'accent sur l'efficacité, en utilisant des structures de données adaptées telles que les dictionnaires et les ensembles pour garantir une **complexité temporelle optimale** en $O(|V| + |E|)$ lors du parcours du graphe (recherche en profondeur, DFS).

Le travail s'est concentré sur deux cas d'étude fondamentaux de la vérification de systèmes concurrents :

1. **Le Sémaphore Binaire** : Le modèle détaillé du sémaphore, ajusté avec la nomenclature N_i, P_i, C_i pour clarifier l'état de chaque processus, a permis de démontrer formellement la vérification de la propriété de **sûreté** la plus critique : l'**Exclusion Mutuelle** ($\neg(C1 \wedge C2)$). Grâce à l'algorithme, nous pouvons confirmer que l'état de violation (les deux processus en section critique) est inaccessible.

2. **Le Compteur (0 à 4) :** Ce deuxième exemple, bien que simple, a illustré la capacité de l'algorithme à **détecter une violation** d'invariant ($\neg \mathbf{MAX}$) et à fournir un **contre-exemple** (le chemin $0 \rightarrow 1 \rightarrow 2 \rightarrow 3$ qui mène bien à l'état de violation : 4), prouvant la complétude de l'approche.

En définitive, ce projet valide non seulement la compréhension théorique des Systèmes de Transition, mais aussi la maîtrise de l'implémentation d'un vérificateur de modèle simple, soulignant l'importance des outils formels pour garantir la fiabilité des systèmes concurrents.